ANEXO TÉCNICO · EVIDENCIA DE EJECUCIÓN

10 / 10 VERIFICADOS

Conflictos de código

10 ejercicios ejecutados, resueltos y **documentados con capturas reales**

Merge, rebase, cherry-pick, stash, binarios, inspección forense, mergetool y estrategias automáticas. Cada práctica incluye su script, su log y la evidencia de validación.

57 PASS · 0 FAIL | 51 capturas reales | Git 2.56.0 | GitHub

Git Bash — evidencia de ejecución

```
$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
$ git status --short
UU app.txt
$ git add app.txt
$ git commit -m "merge: integra feature"
[PASS] el arbol de trabajo queda limpio
[PASS] se conserva el cambio de feature
RESULTADO: 5 PASS, 0 FAIL
```

Ivan <75749748@iegb.edu.pe> · Octubre 2026 · equipo IVAN

github.com/75749748-blip/conflictos-git-ejercicios

10 ejercicios verificados

Este anexo reúne la ejecución de los diez ejercicios de detección y resolución de conflictos. Cada práctica se ejecutó en un repositorio temporal, se resolvió y se validó con comprobaciones automáticas. Las capturas se tomaron de la ventana de comandos de este equipo durante la ejecución; no son recreaciones ni se reconstruyeron desde los logs.

10/10	57	0	51
ejercicios ejecutados	comprobaciones PASS	fallos	capturas reales

Repositorio público:
https://github.com/75749748-blip/conflictos-git-ejercicios
Commit inicial: `df0706c` · Rama `main`

Entorno de prueba. Windows 11 · Git 2.56.0.windows.1 · Git Bash · GitHub CLI 2.102.0 · Chrome headless para generar las capturas. Cada script configura un usuario local de práctica y no modifica la configuración global del equipo.

Cómo se verificó

- Los diez scripts se ejecutan con `bash ejercicios/run-all.sh`; cada uno crea su repositorio temporal en `.work/`.
- Cada script genera el conflicto real, aplica la resolución y ejecuta comprobaciones con `[PASS]` / `[FAIL]`.
- El resultado se guarda en `ejercicios/logs/` y se resume en la última línea: `RESULTADO: N PASS, 0 FAIL`.
- Las capturas se tomaron de la ventana de comandos mientras se ejecutaba cada script; el detalle del procedimiento está en `build/real-captures/README.md`.

Resultados

#	Ejercicio	Operación	Resultado	Evidencia
01	Conflicto simple en una línea	merge	5 PASS · 0 FAIL	script · log
02	Varios hunks en un mismo archivo	merge	6 PASS · 0 FAIL	script · log
03	Conflicto modificar/eliminar	merge	4 PASS · 0 FAIL	script · log
04	Conflicto añadir/añadir	merge	5 PASS · 0 FAIL	script · log
05	Conflicto binario	merge	4 PASS · 0 FAIL	script · log
06	Conflicto durante un rebase	rebase	5 PASS · 0 FAIL	script · log
07	Conflicto en un cherry-pick	cherry-pick	5 PASS · 0 FAIL	script · log
08	Conflicto al aplicar un stash	stash pop	4 PASS · 0 FAIL	script · log
09	Inspección forense y mergetool	merge + mergetool	4 PASS · 0 FAIL	script · log
10	Estrategias automáticas, rerere y merge drivers	estrategias	15 PASS · 0 FAIL	script · log

EJERCICIO 01 · MERGE

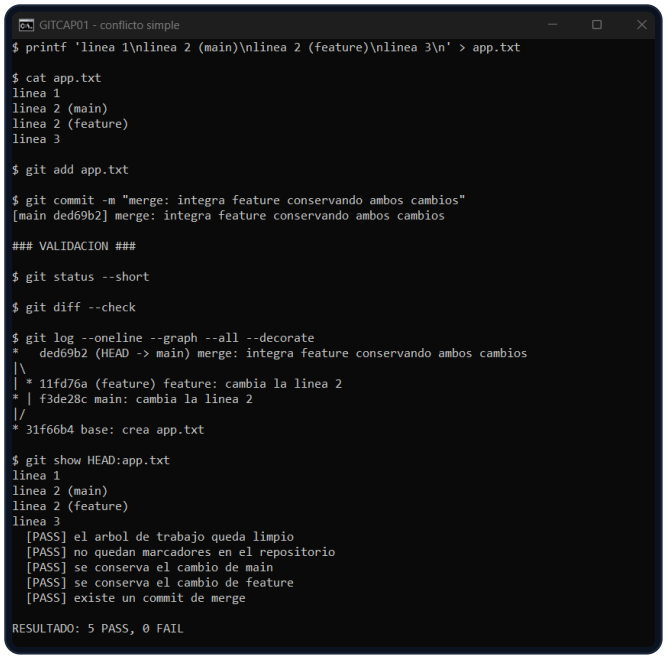
Conflicto simple en una línea

Básica | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Identificar un conflicto UU, leer los marcadores y resolver conservando los dos cambios.

Comandos clave

```
git switch main
git merge feature
git status --short
cat app.txt
git add app.txt
git commit -m "merge: integra feature conservando ambos cambios"
```



Captura 4 de 4 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se conserva el cambio de main
- se conserva el cambio de feature
- existe un commit de merge

> **Lección.** Git detiene la integración porque las dos ramas modificaron la misma línea. La decisión es humana: aquí se conservan ambas intenciones.

Evidencia: script · log completo · capturas

EJERCICIO 02 · MERGE

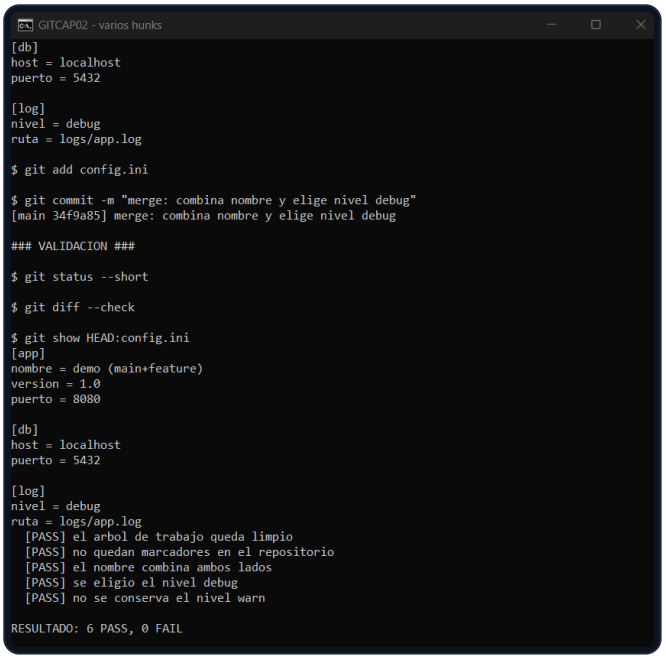
Varios hunks en un mismo archivo

Intermedia | 6 PASS · 0 FAIL | 6 comprobaciones

Objetivo. Resolver dos bloques de conflicto de forma independiente, con una decisión distinta en cada uno.

Comandos clave

```
git merge feature
git status --short
git diff
cat config.ini
git add config.ini
git commit -m "merge: combina nombre y elige nivel debug"
```



Captura 5 de 5 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

6 comprobaciones PASS · 0 FAIL

- hay dos regiones de conflicto
- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el nombre combina ambos lados
- se eligio el nivel debug
- no se conserva el nivel warn

Lección. Un archivo puede contener varios conflictos. Se resuelven por separado: no conviene aplicar --ours o --theirs a todo el archivo.

Evidencia: script · log completo · capturas

EJERCICIO 03 · MERGE

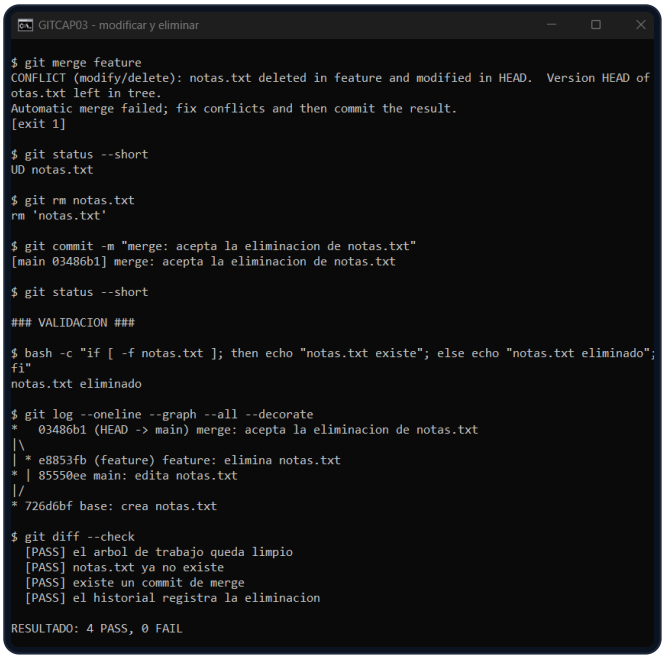
Conflicto modificar/eliminar

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

Objetivo. Decidir entre conservar un archivo editado o aceptar su eliminación, probando las dos soluciones.

Comandos clave

```
git merge feature
git status --short
cat notas.txt
git add notas.txt          # Solución A: conservar
git rm notas.txt           # Solución B: eliminar
git commit -m "merge: acepta la eliminacion de notas.txt"
```



Captura 5 de 5 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- notas.txt ya no existe
- existe un commit de merge
- el historial registra la eliminacion

Lección. El estado UD obliga a una decisión de producto: ¿el archivo debe existir? Se probaron conservar y eliminar, cada una en su propio merge.

Evidencia: script · log completo · capturas

EJERCICIO 04 · MERGE

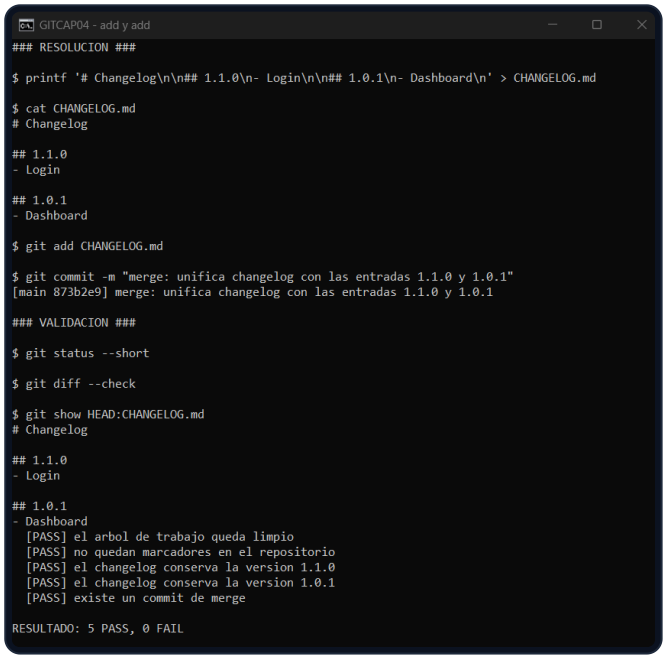
Conflicto añadir/añadir

Intermedia | 5 PASS · 0 FAIL | 5 comprobaciones

Objetivo. Combinar dos archivos nuevos creados de forma independiente con el mismo nombre.

Comandos clave

```
git merge feature
git status --short          # AA CHANGELOG.md
git ls-files -u
cat CHANGELOG.md
git add CHANGELOG.md
git commit -m "merge: unifica changelog con las entradas 1.1.0 y 1.0.1"
```



Captura 4 de 4 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el changelog conserva la version 1.1.0
- el changelog conserva la version 1.0.1
- existe un commit de merge

Lección. En un AA no hay una base común para el archivo. Git muestra los dos documentos completos y deja la unificación al criterio humano.

Evidencia: script · log completo · capturas

EJERCICIO 05 · MERGE

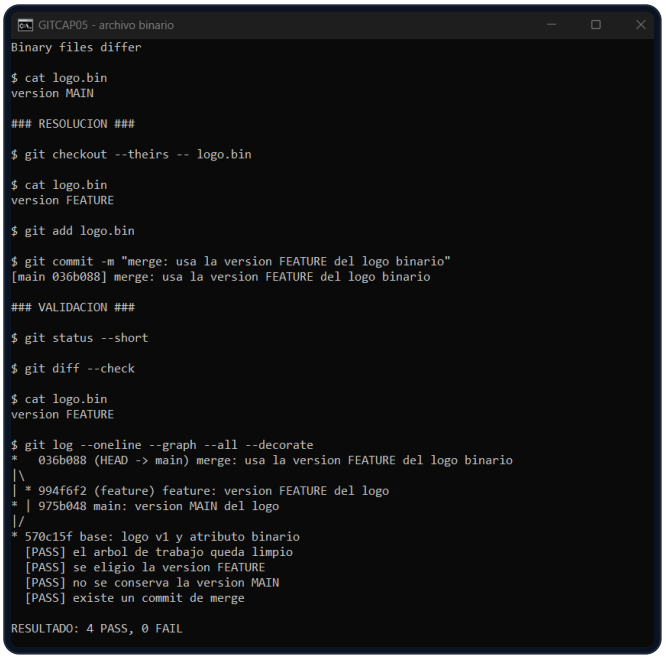
Conflicto binario

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

Objetivo. Resolver un archivo tratado como binario, donde no existen marcadores de texto.

Comandos clave

```
git merge feature
git status --short          # UU logo.bin
git diff                    # Cannot merge binary files
git checkout --theirs -- logo.bin
git add logo.bin
git commit -m "merge: usa la version FEATURE del logo binario"
```



Captura 4 de 4 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- se eligio la version FEATURE
- no se conserva la version MAIN
- existe un commit de merge

Lección. En binarios no se edita el conflicto: se elige una versión completa. Para conservar ambas, hay que renombrar una antes de integrar.

Evidencia: script · log completo · capturas

EJERCICIO 06 · REBASE

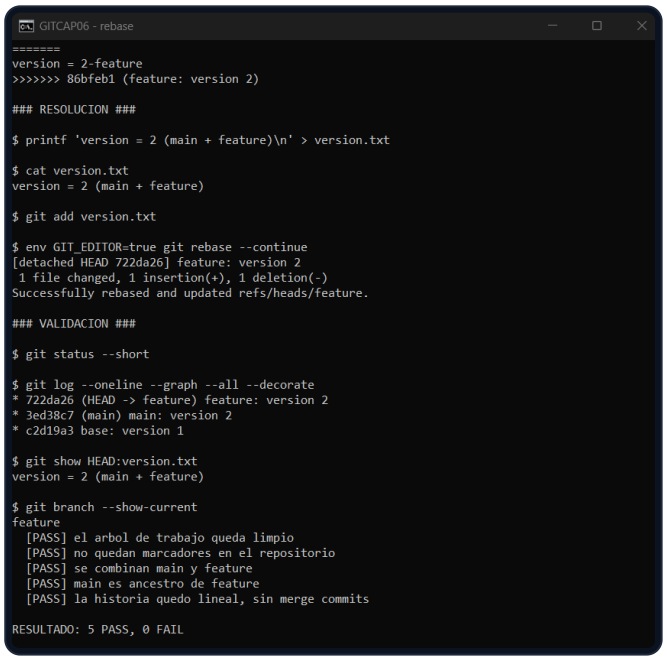
Conflicto durante un rebase

Avanzada | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Resolver un conflicto durante un rebase y comprobar que la historia queda lineal, sin merge commit.

Comandos clave

```
git switch feature
git rebase main
git status
git show :2:version.txt     # main (ours en rebase)
git show :3:version.txt     # feature (theirs en rebase)
git add version.txt
GIT_EDITOR=true git rebase --continue
```



Captura 4 de 4 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan main y feature
- main es ancestro de feature
- la historia quedo lineal, sin merge commits

> **Lección.** En rebase, ours (:2:) es la rama base y theirs (:3:) es el commit que se está reaplicando. La respuesta es la inversa de un merge.

Evidencia: script · log completo · capturas

EJERCICIO 07 · CHERRY-PICK

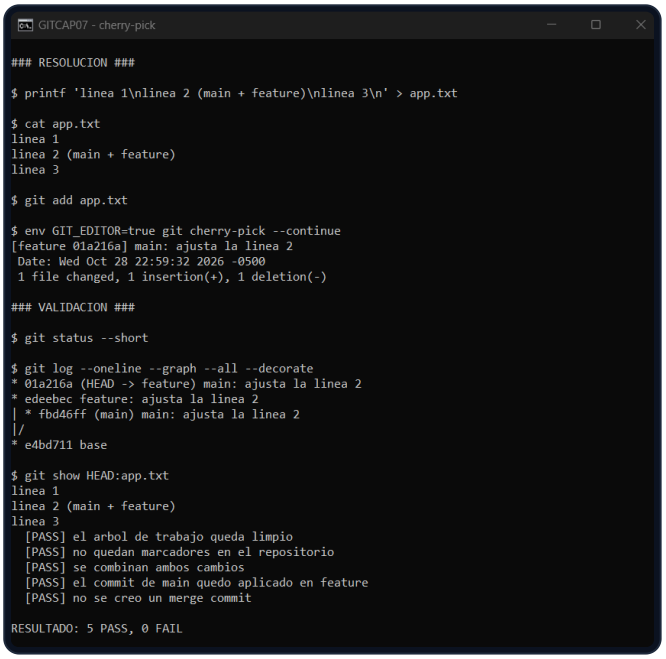
Conflicto en un cherry-pick

Avanzada | 5 PASS · 0 FAIL | 5 comprobaciones

Objetivo. Aplicar un commit concreto de otra rama y resolver el conflicto que genera.

Comandos clave

```
HASH=$(git rev-parse main)
git switch feature
git cherry-pick $HASH
git status
git add app.txt
GIT_EDITOR=true git cherry-pick --continue
```



Captura 5 de 5 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan ambos cambios
- el commit de main quedo aplicado en feature
- no se creo un merge commit

Lección. Cherry-pick reaplica un cambio sobre otra base. El conflicto aparece porque el parche ya no encaja; se resuelve y se continúa con --continue.

Evidencia: script · log completo · capturas

EJERCICIO 08 · STASH POP

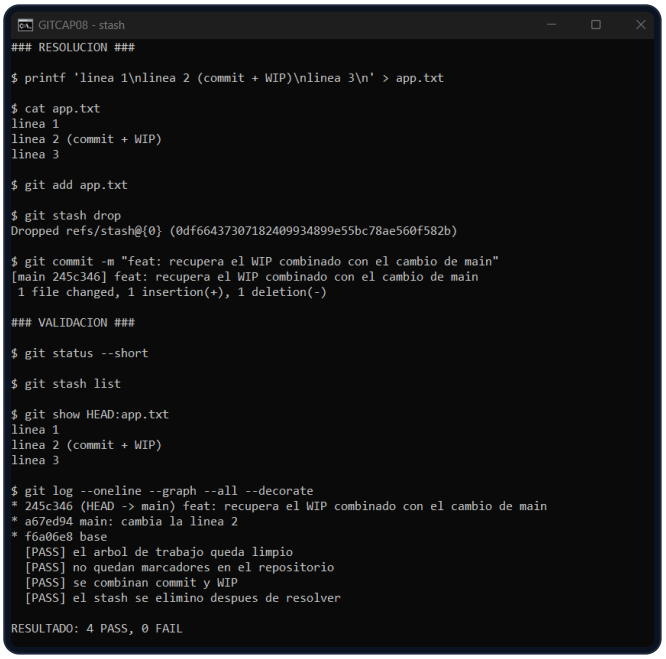
Conflicto al aplicar un stash

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Recuperar trabajo guardado temporalmente cuando el árbol ya cambió.

Comandos clave

```
git stash push -m "wip: linea 2"
git commit -am "main: cambia la linea 2"
git stash pop
git status --short
git add app.txt
git stash drop
git commit -m "feat: recupera el WIP combinado con el cambio de main"
```



Captura 4 de 4 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan commit y WIP
- el stash se elimino despues de resolver

> **Lección.** Si stash pop conflictúa, el stash no se elimina. Se resuelve, se marca el archivo, se hace drop y se confirma el resultado.

Evidencia: script · log completo · capturas

EJERCICIO 09 · MERGE + MERGETOOL

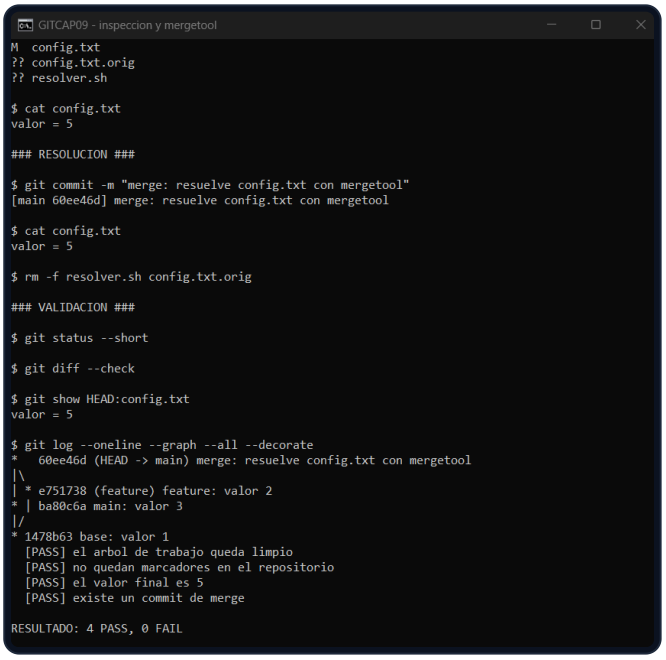
Inspección forense y mergetool

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Usar las etapas del índice, el estilo diff3, git log --merge y una herramienta de merge para resolver con información completa.

Comandos clave

```
git merge feature
git show :1:config.txt    # base
git show :2:config.txt    # ours
git show :3:config.txt    # theirs
git log --merge --oneline -- config.txt
git mergetool --tool=resolver
git commit -m "merge: resuelve config.txt con mergetool"
```



Captura 6 de 6 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el valor final es 5
- existe un commit de merge

> **Lección.** Antes de editar, reconstruye las tres versiones. La base (|||||||) explica qué cambió cada lado y evita elegir a ciegas.

Evidencia: script · log completo · capturas

EJERCICIO 10 · ESTRATEGIAS

Estrategias automáticas, rerere y merge drivers

Avanzada | 15 PASS · 0 FAIL | 15 comprobaciones

> **Objetivo.** Comparar las herramientas que reducen el trabajo manual y entender qué se conserva y qué se descarta en cada una.

Comandos clave

```
git merge -X ours feature       # conserva hunks no conflictivos
git merge -s ours feature       # descarta todo el árbol de feature
git config rerere.enabled true
git config merge.ours.driver true
git merge feature-union         # .gitattributes: merge=union
```



Captura 10 de 10 — cierre de la sesión real, con las comprobaciones y el resultado final. La secuencia completa está en el repositorio.

Validación automática

15 comprobaciones PASS · 0 FAIL

- -X ours conserva la linea de main
- -X ours conserva nueva.txt de feature
- -s ours conserva la linea de main
- -s ours descarta nueva.txt
- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- rerere aplico la resolucion combinada
- existe cache de rerere
- union conserva lock-main
- union conserva lock-b
- union no deja marcadores
- merge=ours conserva main
- merge=ours no aplica feature
- el arbol de trabajo queda limpio (drivers)
- no quedan marcadores (drivers)

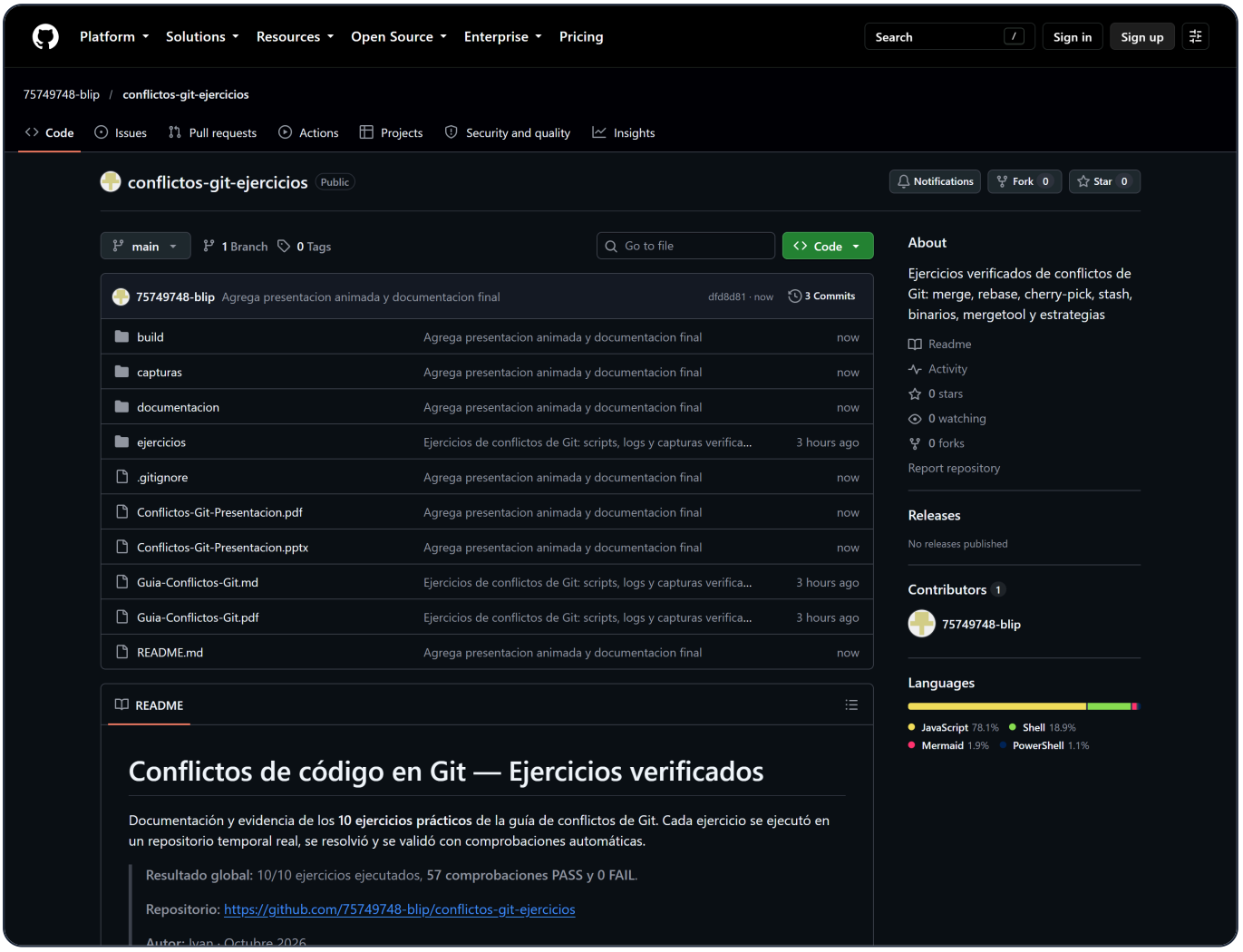
> **Lección.** Ninguna estrategia automática avisa de lo que descartó. -X ours fusiona; -s ours ignora el árbol ajeno; union concatena; ours oculta cambios.

Evidencia: script · log completo · capturas

CIERRE · EVIDENCIA DE PUBLICACIÓN

Repositorio en GitHub

Los scripts, logs, capturas, la guía y la presentación están publicados en el repositorio. La siguiente captura corresponde a la página pública después de publicar la guía, la presentación y la documentación.



URL: https://github.com/75749748-blip/conflictos-git-ejercicios
Commit inicial: df0706c · **Rama:** main · **Visibilidad:** pública

Declaración de verificación. Los diez ejercicios fueron ejecutados en este equipo con Git 2.56.0.windows.1. Los logs muestran 57 comprobaciones PASS y 0 FAIL. Las capturas se tomaron de la consola durante la ejecución. El repositorio de GitHub contiene el código y la evidencia para que cualquier persona pueda repetir el proceso.

Documento generado el 28 de octubre de 2026. Autor: Ivan. Caso de estudio: *Detección y resolución de conflictos de código*.